A aprendizagem de Máquina - II

Atividade para Compensação de Aula (19.09.2026)
Prof. Luis Claudio

Débora Kocks Nogueira – 6º CDN

ATENÇÃO:

Após a data limite, haverá desconto de uma falta por dia de atraso.

Situação-Problema 01 — Configuração e treinamento de uma MLP

Uma empresa deseja utilizar uma rede neural para classificar clientes entre duas categorias. Após a preparação dos dados, cada cliente é representado por **20 atributos numéricos**. A equipe construiu inicialmente uma MLP com duas camadas ocultas, contendo **64 e 32 neurônios**, ambas utilizando ReLU, e uma camada de saída com **1 neurônio e Sigmoid**.

O treinamento foi configurado com batch_size = 128, epochs = 40, learning_rate = 0,001 e otimizador Adam. Durante os testes, a equipe percebeu que diferentes configurações produzem resultados distintos e decidiu avaliar alterações tanto na arquitetura quanto no processo de treinamento.

Questões

1) Identifique, entre as configurações apresentadas, quais correspondem a hiperparâmetros de arquitetura e quais correspondem a hiperparâmetros de treinamento. Explique também por que pesos e biases não pertencem a nenhuma dessas duas categorias.

Na MLP apresentada, os 20 atributos numéricos são as entradas da rede. Já a quantidade de camadas e de neurônios faz parte da arquitetura. Nesse caso, temos duas camadas ocultas, com 64 e 32 neurônios, respectivamente. As funções de ativação ReLU nas camadas ocultas e Sigmoid na saída também fazem parte da configuração da arquitetura.

Já os valores de batch_size = 128, epochs = 40 e learning_rate = 0,001 são relacionados ao treinamento. O Adam também é o otimizador utilizado durante o treinamento.

Os pesos e biases não são hiperparâmetros, porque não são definidos manualmente como uma configuração do modelo. Eles são parâmetros que a própria rede aprende e ajusta durante o treinamento, de acordo com os dados.

2) A equipe pretende testar diferentes quantidades de camadas ocultas, neurônios e funções de ativação. Explique por que essas escolhas precisam ser avaliadas experimentalmente e como elas podem afetar a capacidade de representação da rede.

Essas escolhas precisam ser avaliadas experimentalmente porque não existe uma arquitetura que seja ideal para todos os problemas.

A quantidade de camadas ocultas influencia a capacidade da rede de aprender relações mais complexas. Uma rede com poucas camadas pode não conseguir representar adequadamente determinados padrões, enquanto uma rede muito grande pode ficar mais complexa e apresentar maior possibilidade de overfitting.

A quantidade de neurônios também influencia a capacidade de representação. Mais neurônios permitem que a rede aprenda uma quantidade maior de padrões, mas

aumentar demais essa quantidade pode tornar o modelo desnecessariamente complexo.

As funções de ativação também são importantes porque introduzem não linearidade na rede. Funções como a ReLU permitem que a MLP aprenda relações que não poderiam ser representadas apenas por operações lineares.

Por isso, diferentes combinações de camadas, neurônios e funções de ativação devem ser testadas e comparadas experimentalmente para verificar qual configuração apresenta melhor desempenho para os dados utilizados.

3) Considere agora alterações no batch_size, no número de épocas e no learning_rate. Explique como cada uma dessas alterações pode modificar o processo de treinamento, incluindo quantidade de atualizações, estabilidade do gradiente, velocidade de aprendizado e possibilidade de underfitting ou overfitting.

O batch_size indica quantos exemplos são utilizados antes de a rede fazer uma atualização dos seus parâmetros. Com um batch menor, acontecem mais atualizações durante uma época, mas o gradiente pode apresentar mais variação. Com um batch maior, as atualizações tendem a ser mais estáveis, mas acontecem em menor quantidade por época.

O número de epochs indica quantas vezes a rede passa pelos dados de treinamento. Se forem utilizadas poucas épocas, a rede pode não ter tempo suficiente para aprender os padrões, causando underfitting. Por outro lado, utilizar épocas demais pode fazer a rede se adaptar muito aos dados de treinamento, aumentando a possibilidade de overfitting.

O learning_rate define o tamanho dos passos realizados durante as atualizações. Um valor muito baixo pode deixar o aprendizado muito lento. Já um valor muito alto pode fazer com que a rede tenha dificuldade para encontrar uma boa solução, podendo deixar o treinamento instável.

Por isso, esses valores precisam ser ajustados de acordo com o problema e avaliados durante os testes.

4) Explique, de forma integrada, o papel da função de perda, do gradiente, do learning rate e do otimizador durante o treinamento. Deixe claro por que o gradiente não é um hiperparâmetro.

Batch_size determina quantos exemplos são utilizados antes de uma atualização dos parâmetros da rede.

- Um batch_size menor gera mais atualizações durante cada época. As atualizações podem apresentar mais variação, deixando o gradiente menos estável, mas podem permitir um aprendizado mais frequente.

- Um batch_size maior gera menos atualizações por época e tende a produzir gradientes mais estáveis, porém pode exigir mais memória e nem sempre apresentar o melhor aprendizado.

Epochs representam quantas vezes o modelo passa pelo conjunto de treinamento.

- Poucas épocas podem fazer com que o modelo ainda não tenha aprendido suficientemente os padrões, causando underfitting.
- Aumentar as épocas permite mais tempo para o modelo aprender.
- Porém, épocas demais podem fazer o modelo se adaptar excessivamente aos dados de treinamento, aumentando a possibilidade de overfitting.

Learning rate determina o tamanho do passo utilizado para atualizar os pesos.

- Um learning rate muito pequeno faz o aprendizado acontecer mais lentamente e pode exigir muitas épocas.
- Um learning rate muito alto pode fazer o modelo dar passos muito grandes, dificultando a convergência ou até fazendo com que o treinamento fique instável.
- Um valor adequado permite que a rede aprenda de maneira mais eficiente e estável.

Portanto, batch_size, epochs e learning_rate estão diretamente relacionados à forma como o modelo aprende, e seus valores precisam ser ajustados experimentalmente.

Situação-Problema 02 — Classificação de imagens utilizando CNN

Uma indústria deseja classificar imagens coloridas de produtos para identificar possíveis defeitos. Cada imagem possui dimensão **32 × 32 × 3**. A primeira camada convolucional utiliza **16 filtros 3 × 3**, stride 1, padding same e função ReLU. A rede possui posteriormente outra camada convolucional, uma operação de Max Pooling e, antes das camadas densas, uma operação Flatten.

Durante o desenvolvimento, alguns integrantes da equipe confundiram conceitos como canais, filtros, mapas de características e quantidade de parâmetros, além de questionarem por que utilizar uma CNN em vez de simplesmente transformar toda a imagem em um vetor e utilizar uma MLP.

Questões

5) Explique como um filtro 3 × 3 atua sobre uma imagem RGB. Indique a dimensão completa de cada filtro, como os três canais participam do cálculo e quantos mapas de características são produzidos pelos 16 filtros.

Uma imagem RGB tem 3 canais (vermelho, verde, azul). Um filtro 3×3 aplicado sobre ela não é plano, tem a dimensão 3×3×3 (altura × largura × canais). Para cada posição da imagem, o filtro multiplica seus pesos pelos pixels dos 3 canais simultaneamente e soma tudo, gerando um único valor de saída.

Cada um dos 16 filtros produz um mapa de características independente. Portanto, ao final da primeira camada convolucional, temos 16 mapas de características (um por filtro), cada um representando um padrão diferente aprendido pelo filtro.

6) Calcule a quantidade de parâmetros treináveis dessa primeira camada convolucional. Em seguida, explique por que essa quantidade não aumenta quando a altura e a largura da imagem aumentam, desde que sejam mantidos a quantidade e o tamanho dos filtros e a quantidade de canais.

Cada filtro possui:

3 × 3 × 3 = 27 pesos

Como existem 16 filtros:

27 × 16 = 432 pesos

Além dos pesos, cada filtro possui 1 bias. Como são 16 filtros:

16 biases

Então, o total de parâmetros treináveis é:

432 + 16 = 448 parâmetros

Portanto, a primeira camada convolucional possui 448 parâmetros treináveis.

Essa quantidade não depende diretamente da altura e da largura da imagem porque os filtros possuem tamanho fixo. Se a imagem aumentar de, por exemplo, 32 × 32 para 64 × 64, os mesmos filtros 3 × 3 × 3 continuarão sendo utilizados.

O que aumenta nesse caso é a quantidade de posições em que o filtro será aplicado e, consequentemente, o tamanho dos mapas de características, mas não a quantidade de parâmetros do filtro.

7) Explique como stride e padding influenciam as dimensões espaciais dos mapas de características. Em sua resposta, diferencie same e valid.

O stride indica quantos pixels o filtro avança a cada movimentação pela imagem.

Com stride 1, o filtro se movimenta de um em um pixel, permitindo analisar praticamente todas as posições da imagem. Se o stride for maior, o filtro pula mais posições e o mapa de características tende a ficar menor.

O padding é utilizado para adicionar pixels ao redor da imagem antes da aplicação do filtro.

No padding same, são adicionadas bordas para que a dimensão espacial da saída seja mantida, considerando o stride utilizado. No caso apresentado, com stride 1, a imagem de entrada de 32 × 32 continua gerando um mapa de características de 32 × 32.

Já no padding valid, nenhum preenchimento é adicionado. Por isso, como o filtro precisa estar completamente dentro da imagem, a dimensão espacial da saída fica menor.

8) A primeira camada produz 16 mapas de características, que são recebidos pela segunda camada convolucional. Explique qual será a profundidade de cada filtro da segunda camada, quantos mapas de características serão produzidos pelos seus 32 filtros e qual é a função do Flatten antes das camadas densas.

A primeira camada produziu 16 mapas de características, que se tornam os 16 canais de entrada para a segunda camada. Por isso:

- Cada filtro da segunda camada tem profundidade 16 ->
  dimensão 3×3×16 (assumindo filtros 3×3).
- Com 32 filtros, a segunda camada produz 32 mapas de características.

Função do Flatten: As camadas convolucionais produzem tensores 3D (altura × largura × canais). As camadas densas (fully connected) esperam vetores 1D.
O Flatten simplesmente "estica" esse tensor 3D em um único vetor, sem perder nenhuma informação , é apenas uma mudança de forma para conectar a parte convolucional à parte densa da rede.